Database Users, Roles, Privileges, and Access Control

Group 2(PARK IT)

Section	Contents
Cover and links	**GROUP 12: Amoranto, Denielle; Eurescial, Eunice; Francisco, Helix** **Project Assigned:** On-the-Job Training (OJT) Record Management Application **Repository:** **Link:https://github.com/millerdevera6-glitch/park-it-database.git** **Our Group Repository:**
Projects set up	**The Park-It Management System repository was used as the basis for implementing the database access-control plan. The project uses PostgreSQL to store and manage information related to parking zones, vehicles, transactions, payments, and staff accounts.** **The first step was to examine the existing project files and understand the database structure. The original database schema was reviewed to identify the tables and other database objects that needed access restrictions.** **The database used for the activity was named `grp_parkitdb`. The existing database structure was retained, and the access-control implementation was performed separately to avoid changing the original system functionality.** **The following database tables were identified:** • **parking_zone – Stores information about parking areas and their available capacity.** • **payment – Contains records of parking payments.** • **staff_user – Stores staff account information.** • **transaction – Records vehicle entry and exit activities.** • **vehicle – Contains registered vehicle information.** **After reviewing the database structure, a separate folder named `access_control` was prepared to organize the SQL scripts and access-control documentation.** **The following files were created:** 1. `01_create_roles.sql`

	2. `02_grant_privileges.sql` 3. `03_verify_permissions.sql` 4. `access_control_plan.md` **These files were organized to make the implementation easier to understand and maintain.**
Access-control plan	**The access-control plan was created to determine which database operations should be allowed for each role. The implementation follows the principle of least privilege, meaning that each role should only receive the permissions needed for its responsibilities.** **Three PostgreSQL group roles were established.** **Administrator Role (parkit_admin_grp)** **The administrator role is responsible for managing and maintaining the database. It receives full privileges on existing tables and sequences within the public schema.** **Its responsibilities include managing parking zone information, maintaining vehicle records, handling transaction and payment records, and managing staff-related information.** **Runtime Role (parkit_runtime_grp)** **The runtime role is intended for normal application operations. It allows the application to retrieve, add, update, and delete database records when performing its functions.** **Its responsibilities include accessing vehicle information, recording parking transactions, processing payment records, and updating parking zone information.** **Read-only Role (parkit_readonly_grp)** **The read-only role is intended for users who only need to view database information. It receives SELECT privileges but cannot insert, update, or delete records.** **Its purpose is to allow users to view parking information without making unauthorized changes.**

<table>
<tr>
<td></td>
<td>
Access-Control Matrix
<table>
<tr><th>Database Operations</th><th>Admin</th><th>Runtime</th><th>Read-only</th></tr>
<tr><td>SELECT</td><td>Allowed</td><td>Allowed</td><td>Allowed</td></tr>
<tr><td>INSERT</td><td>Allowed</td><td>Allowed</td><td>Denied</td></tr>
<tr><td>UPDATE</td><td>Allowed</td><td>Allowed</td><td>Denied</td></tr>
<tr><td>DELETE</td><td>Allowed</td><td>Allowed</td><td>Denied</td></tr>
<tr><td>Sequence Access</td><td>Full</td><td>Usage and Select</td><td>Not granted</td></tr>
</table>
The three roles were created with the <code>NOLOGIN</code> attribute. This means they function as permission groups and must be assigned to actual login accounts before individual users or application accounts can use their permissions.
</td>
</tr>
<tr>
<td>SQL scripts summary</td>
<td>
Three SQL scripts were prepared to implement and verify the access-control configuration.

File 1: 01_create_roles.sql

The first SQL script was used to create the three PostgreSQL group roles.

The script checks whether each role already exists before creating it. This prevents errors when the script is executed more than once.

The following roles were created:

<code>parkit_admin_grp</code>
<code>parkit_runtime_grp</code>
<code>parkit_readonly_grp</code>

After executing the script through pgAdmin 4, a verification query was
</td>
</tr>
</table>

	used to display the created roles from the PostgreSQL system catalog. **File 2: 02_grant_privileges.sql** **The second SQL script was used to assign the required database permissions to each role.** **First, the three roles were given permission to connect to the `grp_parkitdb` database and access the public schema.** **The administrator role received full privileges on existing tables and sequences. The runtime role received SELECT, INSERT, UPDATE, and DELETE permissions on existing tables, together with the required sequence permissions. The read-only role received SELECT privileges only.** **The script also included default privileges for future tables and sequences created by the `postgres` role. This helps maintain consistent permissions when new database objects are added under that owner.** **File 3: 03_verify_permissions.sql** **The third SQL script was used to check whether the assigned permissions were correct.** **The script contains queries that display the created roles, list their table privileges, and test specific permissions using PostgreSQL's `has_table_privilege()` function.** **The verification focused on checking whether:** • **The administrator can delete records.** • **The runtime role can insert records.** • **The read-only role can select records.** • **The read-only role cannot insert records.** **These checks help confirm that each role follows the intended access-control plan.**
Verification results	**After preparing the three SQL scripts, they were executed in the PostgreSQL Query Tool through pgAdmin 4.** **The scripts were executed in the correct order to avoid**

permission-related errors.

First Verification: Role Creation

The first script was executed to create the three group roles. The PostgreSQL system catalog was queried to confirm that the roles existed.

Expected result: All three roles should appear in the query results.

Second Verification: Privilege Assignment

The second script was executed to assign the required database and schema privileges. The execution messages were checked to determine whether the commands completed successfully.

Expected result: Each role should receive the permissions defined in the access-control plan.

Third Verification: Permission Checking

The third script was executed to verify the permissions using PostgreSQL queries.

The expected results were:

Verification	Expected Result
Admin can delete records	TRUE
Runtime can insert records	TRUE
Read-only can select records	TRUE
Read-only can insert records	FALSE

The expected results demonstrate that the administrator and runtime roles have their intended database permissions, while the read-only role is restricted to viewing records.

Screenshots:

	• **Screenshot 1:** `01_create_roles.png` • **Screenshot 2:** `02_grant_privileges.png` • **Screenshot 3:** `03_verify_permissions.png` **The screenshots serve as supporting evidence of the SQL execution and verification process.**
Issues and fixes	**During the preparation and implementation of the access-control configuration, the following possible issues were considered and addressed.** • **Database Name Mismatch: The SQL scripts must be executed inside the correct database. The database name `grp_parkitdb` was used consistently to avoid applying the configuration to the wrong database.** • **Existing Database Roles: Running role creation commands repeatedly may cause errors if the roles already exist. The first SQL script uses an existence check before creating each role.** • **Incorrect Execution Order: Granting permissions before creating the roles can result in errors. To prevent this, the scripts were executed in order, starting with role creation, followed by privilege assignment, and then verification.** • **Insufficient Database Permissions: Some PostgreSQL commands require appropriate administrative privileges. The scripts were executed using the PostgreSQL `postgres` account to ensure that the necessary permissions were available.** • **Future Database Objects: Newly created tables and sequences may not automatically receive the same permissions as existing objects. Default privileges were included to help address this issue for objects created by the `postgres` role in the public schema.** • **Read-only Permission Restrictions: The read-only role should not be able to modify database records. The verification script checks that SELECT is allowed while INSERT is denied.**
Feedback to owners	**The Park-It Management System already has a database structure that supports its main parking management functions. However, implementing role-based access control can further improve the security and organization of database operations.**

	The following recommendations are provided to the project owners: ● **Maintain separate database roles for administrators and normal application operations.** ● **Avoid using administrator-level database credentials for regular application activities.** ● **Assign actual login accounts to the appropriate group roles.** ● **Use strong passwords and protect database credentials.** ● **Review database permissions whenever new tables or features are added.** ● **Restrict access to sensitive payment and staff information.** ● **Regularly check database roles and privileges to prevent unnecessary access.** **These recommendations can help improve the security and maintainability of the Park-It Management System.** ---
AI usage declaration	**Artificial Intelligence (AI) tools were used as a supplementary resource during the activity to assist in understanding PostgreSQL role creation, database privileges, SQL syntax, and documentation preparation.** **AI assistance was used to help organize the SQL scripts, explain the purpose of each access-control role, and identify possible implementation issues and their corresponding solutions.** **The SQL scripts were reviewed and executed using PostgreSQL through pgAdmin 4. The student remains responsible for checking the correctness of the implementation, understanding the assigned permissions, and verifying the actual execution results.** --- ## Conclusion **The Activity 4 implementation established a role-based access-control structure for the Park-It Management System using PostgreSQL. Three group roles were prepared to separate administrative operations, normal**

	application activities, and read-only access. **Through the creation of roles, assignment of privileges, and verification of permissions, the activity demonstrated how database access can be managed according to user responsibilities. This implementation helps protect important parking records and provides a more organized approach to database security.**